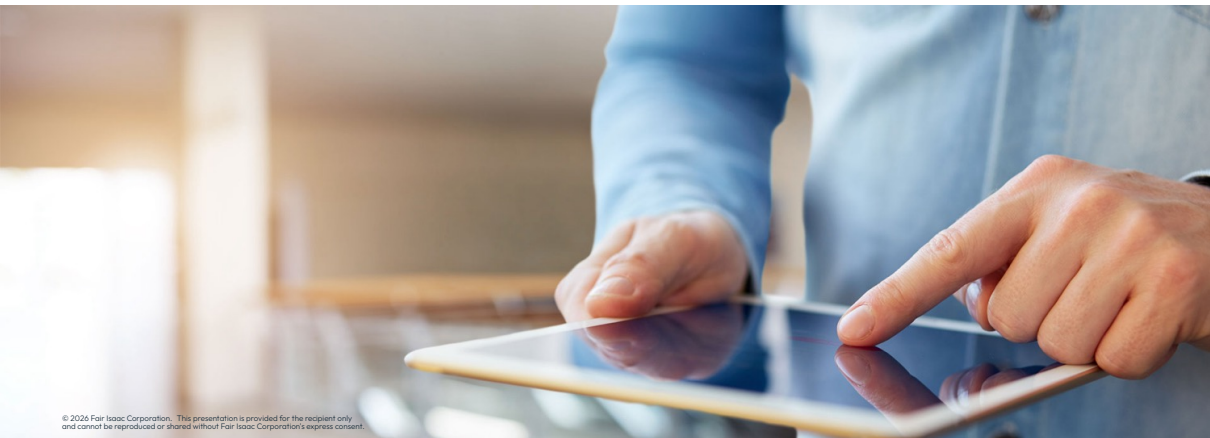




# Hints for efficiency





# Building models efficiently in Python

## Avoid explicit loops

- The Xpress Python module facilitates the use of lists, dictionaries, and sets as arguments in most of its methods:
  - This ensures faster execution by avoiding using explicit loops which usually increase model building times
  - This is especially relevant in large optimization models with multiple calls to functions such as `p.addVariable()` and `p.addConstraint()`

# Avoid explicit loops

- The Xpress Python module facilitates the use of lists, dictionaries, and sets as arguments in most of its methods:
  - This ensures faster execution by avoiding using explicit loops which usually increase model building times
  - This is especially relevant in large optimization models with multiple calls to functions such as `p.addVariable()` and `p.addConstraint()`
- Consider a loop which makes N calls to `p.addConstraint`:

```
x = [p.addVariable() for i in range(N)]  
y = [p.addVariable(vartype=xp.binary) for i in range(N)]  
for i in range(N):  
    p.addConstraint(x[i] <= y[i])
```

- The external loop can be replaced by a single call to `p.addConstraint` with [an inner loop](#):  
`p.addConstraint(x[i] <= y[i] for i in range(N))`

## Use *SciPy* sparse arrays

- Sparse data is a data set where most elements have a value zero:
  - Can be an array like `[1, 0, 2, 0, 0, 3, 0, 0, 0, 0, 0]`
  - Sparse array formats allow building models more efficiently by avoiding iterating over all the elements (including the zeros) of a conventional array
- The *SciPy* package has a module, `scipy.sparse` that provides functions to deal with sparse data

# Use *SciPy* sparse arrays

- Sparse data is a data set where most elements have a value zero:
  - Can be an array like [1, 0, 2, 0, 0, 3, 0, 0, 0, 0, 0]
  - Sparse array formats allow building models more efficiently by avoiding iterating over all the elements (including the zeros) of a conventional array
- The *SciPy* package has a module, `scipy.sparse` that provides functions to deal with sparse data
- The `xp.Dot()` operator supports the most common *SciPy* sparse matrix formats, allowing arrays of sparse expressions and constraints to be constructed efficiently:
  - Can compute the product of a 1-D NumPy array of variables or expressions with a sparse matrix of numbers in CSR or CSC format

```
import numpy as np
from scipy.sparse import csr_matrix
```

```
orig_array = np.array([1, 0, 2, 0, 0, 3, 0, 0, 0, 0, 0])           # sparse np array
scipy_array = csr_matrix(orig_array)                               # convert to scipy sparse array form
p.addConstraint(xp.Dot(scipy_array, var) <= rhs)                  # use with xp.Dot
```

## Using the low-level API functions

- The `problem.load*()` functions provide a low-level interface to the FICO® Xpress Optimizer libraries:
  - Functions: `p.loadLP`, `p.loadMIP`, `p.loadQP`, `p.loadMIQP`, `p.loadQCQP`, `p.loadMIQCQP`
  - Preferable with very large problems and when efficiency in model creation is necessary
  - Can be used to create problems with linear/quadratic constraints, a linear/quadratic objective function, and with continuous/discrete variables

# Using the low-level API functions

- The `problem.load*()` functions provide a low-level interface to the FICO® Xpress Optimizer libraries:
  - Functions: `p.loadLP`, `p.loadMIP`, `p.loadQP`, `p.loadMIQP`, `p.loadQCQP`, `p.loadMIQCQP`
  - Preferable with very large problems and when efficiency in model creation is necessary
  - Can be used to create problems with linear/quadratic constraints, a linear/quadratic objective function, and with continuous/discrete variables
- Consider the following MIQP model built using the high-level functions:

```
import xpress as xp
p = xp.problem(name='myexample')
x = p.addVariable(vartype=xp.integer, name='x1', lb=-10, ub=10)
y = p.addVariable(name='x2')
p.setObjective(x**2 + 2*y)
p.addConstraint(x + 3*y <= 4)
p.addConstraint(7*x + 4*y >= 8)
```



**Hint:** Check other low-level API functions such as `problem.addrows()`, `problem.addcols()`, and `problem.addqmatrix()`

# Using the low-level API functions

- The same problem can be created using `problem.loadMIQP()`, including variable names and their types:

```
p = xp.problem()
p.loadMIQP(probname='myexample',
            rowtype=['L', 'G'],      # constraint senses
            rhs=[4, 8],             # right-hand sides
            rng=None,               # no range rows
            objcoef=[0, 2],         # linear obj. coeff.
            start=[0, 2, 4],        # start pos. of all columns
            rowind=[0, 1, 0, 1],    # row index in each column
            rowcoef=[1, 7, 3, 4],   # coefficients
            lb=[-10,0],             # variable lower bounds
            ub=[10,xp.infinity],    # upper bounds
            objqcoll=[0],           # quadratic obj. terms, column 1
            objqcol2=[0],          # column 2
            objqcoef=[2],           # coeff
            coltype=['I'],          # variable types
            entind=[0])             # index of integer variable
```

# DEMO:

## Building models efficiently using the Python API

- A series of examples of practices for [improving the model building performance](#) when using the Xpress Python API:
  - Using list comprehensions to avoid explicit loops when adding variables and constraints
  - Using NumPy arrays to create variables and constraint and the `xp.Dot()` operator for products of NumPy arrays
  - Using SciPy sparse arrays for large and sparse data sets
  - Using low-level API functions such as `problem.load*()` to build large models efficiently



Thank You

[www.fico.com/optimization](http://www.fico.com/optimization)